

Code Overview

The code implements a recommendation system with three main components:

1. A collisionless embedding table
2. A DeepFM recommendation model
3. A simple recommendation system that ties everything together

Library Imports

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from collections import defaultdict
import datetime
```

These libraries provide the necessary functionality:

- numpy: For numerical operations and random data generation
- torch: PyTorch library for building neural networks
- torch.nn: Neural network components like layers
- torch.nn.functional: Activation functions and operations
- Dataset, DataLoader: For efficient data batching
- defaultdict: An enhanced dictionary data structure
- datetime: For tracking when embeddings were last accessed

CollisionlessEmbeddingTable Class

This class maintains unique embeddings for each feature (user or item) without hash collisions.

Initialization

```
class CollisionlessEmbeddingTable:
    def __init__(self, embedding_dim=16, expiration_days=30):
        self.embedding_dim = embedding_dim
        self.expiration_days = expiration_days
        self.embedding_dict = {} # feature_id -> (embedding, last_access_time)
```

- embedding_dim: Size of each embedding vector (default: 16)
- expiration_days: Days after which unused embeddings expire (default: 30)
- embedding_dict: Dictionary storing embeddings and their last access time

Lookup Method

```
def lookup(self, feature_ids):
    """
    Look up embeddings for feature ids, creating new ones if they don't exist.
    Also updates the last access time.
    """
    current_time = datetime.datetime.now()
    embeddings = []

    for feature_id in feature_ids:
        if feature_id in self.embedding_dict:
            embedding, _ = self.embedding_dict[feature_id]
            self.embedding_dict[feature_id] = (embedding, current_time)
        else:
            # Initialize a new embedding for this feature
            embedding = torch.randn(self.embedding_dim) * 0.01
```

```

        self.embedding_dict[feature_id] = (embedding, current_time)

    embeddings.append(embedding)

    return torch.stack(embeddings) if embeddings else torch.zeros((0,
self.embedding_dim))

```

This method:

1. Takes a list of feature IDs
2. For each ID:
 - If it exists in the dictionary, retrieves its embedding and updates access time
 - If it doesn't exist, creates a new random embedding for it
3. Returns a stacked tensor of all embeddings

Update Method

```

def update(self, feature_ids, gradients):
    """
    Update embeddings based on gradients.
    """
    learning_rate = 0.01 # Simplified learning rate

    for i, feature_id in enumerate(feature_ids):
        if feature_id in self.embedding_dict:
            embedding, last_access = self.embedding_dict[feature_id]
            updated_embedding = embedding - learning_rate * gradients[i]
            self.embedding_dict[feature_id] = (updated_embedding, last_access)

```

This method:

1. Takes feature IDs and their gradients (how embeddings should change)
2. Updates each embedding using simple gradient descent
3. Note: This method is not actually used in the main workflow

Expire Old Embeddings Method

```

def expire_old_embeddings(self):
    """
    Remove embeddings that haven't been accessed for a long time.
    """
    current_time = datetime.datetime.now()
    expiration_threshold = current_time -
datetime.timedelta(days=self.expiration_days)

    keys_to_remove = []
    for feature_id, (_, last_access) in self.embedding_dict.items():
        if last_access < expiration_threshold:
            keys_to_remove.append(feature_id)

    for feature_id in keys_to_remove:
        del self.embedding_dict[feature_id]

    return len(keys_to_remove) # Return number of expired embeddings

```

This method:

1. Calculates an expiration threshold based on the current time
2. Identifies embeddings that haven't been accessed since that threshold
3. Removes them from the dictionary

4. Returns the count of removed embeddings

DeepFM Model

This class implements the DeepFM architecture, which combines factorization machines with deep neural networks.

Initialization

```
class DeepFM(nn.Module):
    def __init__(self, field_dims, embedding_dim=16, mlp_dims=(64, 32), dropout=0.2):
        """
        Parameters:
        - field_dims: A list of field dimensions (number of features in each field)
        - embedding_dim: The size of embedding vectors
        - mlp_dims: The hidden layer dimensions for MLP part
        - dropout: Dropout rate
        """
        super(DeepFM, self).__init__()

        self.num_fields = len(field_dims)
        self.embedding_dim = embedding_dim

        # FM component
        self.fm_first_order = nn.ModuleList([
            nn.Embedding(field_dim, 1) for field_dim in field_dims
        ])
        self.fm_second_order = nn.ModuleList([
            nn.Embedding(field_dim, embedding_dim) for field_dim in field_dims
        ])

        # Deep component
        self.mlp_input_dim = self.num_fields * embedding_dim
        self.mlp = nn.Sequential()
        input_dim = self.mlp_input_dim

        for i, dim in enumerate(mlp_dims):
            self.mlp.add_module(f'linear_{i}', nn.Linear(input_dim, dim))
            self.mlp.add_module(f'relu_{i}', nn.ReLU())
            self.mlp.add_module(f'dropout_{i}', nn.Dropout(p=dropout))
            input_dim = dim

        self.mlp.add_module('linear_final', nn.Linear(input_dim, 1))

        # Initialize weights
        for embedding in self.fm_first_order:
            nn.init.normal_(embedding.weight, std=0.01)

        for embedding in self.fm_second_order:
            nn.init.normal_(embedding.weight, std=0.01)
```

The initialization:

1. Sets up embedding tables for the FM component:
 - First-order embeddings (scalar values for each field)
 - Second-order embeddings (vectors for field interactions)
2. Builds a multi-layer perceptron (MLP) for the deep component
3. Initializes all weights with small random values

Forward Method

```
def forward(self, x):
    """
    Forward pass of DeepFM.
    x: input tensor of shape (batch_size, num_fields)
    """
    # FM First Order
    fm_first_order_sum = 0
    for i in range(self.num_fields):
        fm_first_order_sum += self.fm_first_order[i](x[:, i])

    # FM Second Order
    fm_second_order_embeddings = [
        self.fm_second_order[i](x[:, i]) for i in range(self.num_fields)
    ]

    # Sum of squares - square of sum
    sum_of_square = sum(embedding ** 2 for embedding in fm_second_order_embeddings)
    square_of_sum = sum(fm_second_order_embeddings) ** 2
    fm_second_order_sum = 0.5 * (square_of_sum - sum_of_square).sum(1, keepdim=True)

    # Deep component
    deep_input = torch.cat(fm_second_order_embeddings, dim=1)
    deep_out = self.mlp(deep_input)

    # Final prediction
    prediction = fm_first_order_sum + fm_second_order_sum + deep_out
    return torch.sigmoid(prediction.squeeze(1))
```

This method:

1. Computes first-order interactions (linear terms)
2. Computes second-order interactions (pairwise interactions between fields)
 - Uses a mathematical trick (sum of squares - square of sums) for efficiency
3. Passes concatenated embeddings through the deep neural network
4. Combines all three components and applies sigmoid to get a probability

SimpleRecommendationSystem Class

This class orchestrates the entire recommendation process.

Initialization

```
class SimpleRecommendationSystem:
    def __init__(self, num_users, num_items, embedding_dim=16):
        self.num_users = num_users
        self.num_items = num_items
        self.embedding_dim = embedding_dim

        # Use the collisionless embedding table for users and items
        self.user_embedding_table = CollisionlessEmbeddingTable(embedding_dim)
        self.item_embedding_table = CollisionlessEmbeddingTable(embedding_dim)

        # DeepFM model for prediction
        self.model = DeepFM(field_dims=[num_users, num_items],
                             embedding_dim=embedding_dim)
        self.optimizer = torch.optim.Adam(self.model.parameters(), lr=0.001)
        self.criterion = nn.BCELoss()
```

The initialization:

1. Stores basic parameters (number of users, items, embedding dimension)
2. Creates embedding tables for users and items (note: these aren't actually used in the main workflow)
3. Initializes the DeepFM model
4. Sets up an optimizer (Adam) for training
5. Sets up a loss function (Binary Cross Entropy)

Train Method

```
def train(self, user_ids, item_ids, labels, batch_size=64, epochs=5):
    """
    Train the recommendation model.
    """
    dataset = list(zip(user_ids, item_ids, labels))
    data_loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

    self.model.train()
    for epoch in range(epochs):
        total_loss = 0
        for user_batch, item_batch, label_batch in data_loader:
            self.optimizer.zero_grad()

            # Convert to tensors
            user_batch = user_batch.long()
            item_batch = item_batch.long()
            label_batch = label_batch.float()

            # Forward pass
            inputs = torch.stack([user_batch, item_batch], dim=1)
            predictions = self.model(inputs)

            # Calculate loss
            loss = self.criterion(predictions, label_batch)

            # Backward pass and optimize
            loss.backward()
            self.optimizer.step()

            total_loss += loss.item()

        avg_loss = total_loss / len(data_loader)
        print(f"Epoch {epoch+1}/{epochs}, Loss: {avg_loss:.4f}")

        # Expire old embeddings
        num_expired_users = self.user_embedding_table.expire_old_embeddings()
        num_expired_items = self.item_embedding_table.expire_old_embeddings()
        if num_expired_users > 0 or num_expired_items > 0:
            print(f"Expired {num_expired_users} user embeddings and {num_expired_items} item embeddings")
```

This method:

1. Creates a DataLoader to efficiently batch the training data
2. For each epoch:
 - For each batch:
 - Prepares input tensors

- Makes predictions
- Calculates loss
- Updates model weights via backpropagation
- Reports average loss for the epoch
- Expires old embeddings from the embedding tables

Predict Method

```
def predict(self, user_ids, item_ids):
    """
    Make predictions for user-item pairs.
    """
    self.model.eval()
    with torch.no_grad():
        user_tensor = torch.tensor(user_ids).long()
        item_tensor = torch.tensor(item_ids).long()
        inputs = torch.stack([user_tensor, item_tensor], dim=1)
        predictions = self.model(inputs)

    return predictions.numpy()
```

This method:

1. Sets the model to evaluation mode
2. Disables gradient calculation for efficiency
3. Converts user and item IDs to tensors
4. Feeds them through the model
5. Returns the predicted scores as NumPy arrays

Recommend Items Method

```
def recommend_items(self, user_id, top_k=10):
    """
    Recommend top-k items for a user.
    """
    # Generate predictions for all items
    item_ids = list(range(self.num_items))
    user_ids = [user_id] * len(item_ids)
    predictions = self.predict(user_ids, item_ids)

    # Sort items by prediction score
    item_scores = list(zip(item_ids, predictions))
    item_scores.sort(key=lambda x: x[1], reverse=True)

    # Return top-k items
    return [item for item, _ in item_scores[:top_k]]
```

This method:

1. Takes a user ID and the number of recommendations to make
2. Generates prediction scores for all possible items
3. Sorts the items by their prediction scores
4. Returns the top-K items as recommendations

Example Usage

```
if __name__ == "__main__":
    # Sample data
    num_users = 1000
```

```

num_items = 500

# Synthetic data
np.random.seed(42)
user_ids = np.random.randint(0, num_users, size=10000)
item_ids = np.random.randint(0, num_items, size=10000)
labels = np.random.randint(0, 2, size=10000) # Binary labels (click/no-click)

# Create recommendation system
rec_system = SimpleRecommendationSystem(num_users, num_items)

# Convert numpy arrays to torch tensors
user_ids_tensor = torch.tensor(user_ids)
item_ids_tensor = torch.tensor(item_ids)
labels_tensor = torch.tensor(labels)

# Train the model
rec_system.train(user_ids_tensor, item_ids_tensor, labels_tensor, epochs=3)

# Make recommendations for a user
user_id = 42
recommended_items = rec_system.recommend_items(user_id, top_k=5)
print(f"Top 5 recommended items for user {user_id}: {recommended_items}")

```

This section:

1. Creates synthetic data with 1,000 users and 500 items
2. Generates 10,000 random user-item interactions
3. Initializes the recommendation system
4. Converts the data to PyTorch tensors
5. Trains the model for 3 epochs
6. Generates and prints top-5 recommendations for user 42

Performance Bottleneck and ANN Solution

The current implementation has a major bottleneck in the `recommend_items` method. For each user, it computes scores for **all** items, which becomes inefficient as the number of items grows.

This is where Approximate Nearest Neighbors (ANN) comes in:

1. During training, we build an index of item embeddings
2. For recommendations, we:
 - Extract the user's embedding
 - Use ANN to find similar items in embedding space (candidate generation)
 - Only score these candidates with the full model (re-ranking)

This two-stage approach can reduce complexity from $O(n)$ to approximately $O(\log n)$, making recommendations much faster with large item catalogs.